Predict NYC Airbnb Prices — Complete Python Code

Google Colab code. Upload AB_NYC_2019.csv when prompted. Model metrics are calculated when you run the notebook; no scores are invented here.

```
# Predict New York City Airbnb Prices using Linear Regression
# Google Colab: upload AB_NYC_2019.csv when prompted.

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from io import BytesIO
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# 1. Load dataset
uploaded = files.upload()
filename = next(iter(uploaded))
df = pd.read_csv(BytesIO(uploaded[filename]))

# 2. EDA / data audit
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("\\nData types:\\n", df.dtypes)
print("\\nMissing values:\\n", df.isnull().sum())
print("\\nMissing percentage:\\n", (df.isnull().mean() * 100).round(2))
print("\\nNumeric summary:\\n", df.describe())

df["neighbourhood_group"].value_counts().plot(kind="bar", title="Listings by Neighbourhood Group")
plt.xlabel("Neighbourhood Group"); plt.ylabel("Listings"); plt.tight_layout(); plt.show()
df["room_type"].value_counts().plot(kind="bar", title="Listings by Room Type")
plt.xlabel("Room Type"); plt.ylabel("Listings"); plt.tight_layout(); plt.show()

# 3. Clean data
df = df.dropna(subset=["price"])
df = df[df["price"] > 0]
if "reviews_per_month" in df.columns:
    df["reviews_per_month"] = df["reviews_per_month"].fillna(0)
if "last_review" in df.columns:
    df["last_review"] = df["last_review"].fillna("No reviews")

drop_cols = ["id", "name", "host_name", "host_id", "last_review",
             "latitude", "longitude", "neighbourhood"]
df = df.drop(columns=[c for c in drop_cols if c in df.columns])
for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include=["object"]).columns:
    if df[col].isna().any():
        df[col] = df[col].fillna(df[col].mode()[0])

# 4. View distributions and handle outliers
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(df["price"], bins=40); axes[0].set_title("Price Distribution")
axes[1].hist(df["minimum_nights"], bins=40); axes[1].set_title("Minimum Nights")
plt.tight_layout(); plt.show()
```

Python Code (continued)

```
price_limit = df["price"].quantile(0.95)
df = df[(df["price"] <= price_limit) & (df["minimum_nights"] < 365)]

# 5. Encode categorical features
X = df.drop(columns=["price"])
y = df["price"]
X = pd.get_dummies(X, columns=["neighbourhood_group", "room_type"],
                   drop_first=True, dtype=int)

# 6. Split and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# 7. Evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("\\nR2:", round(r2_score(y_test, y_pred), 3))
print("MAE:", round(mean_absolute_error(y_test, y_pred), 2))
print("RMSE:", round(rmse, 2))

# 8. Actual vs predicted plot
plt.figure(figsize=(6, 5))
plt.scatter(y_test, y_pred, alpha=0.35)
low = min(y_test.min(), y_pred.min()); high = max(y_test.max(), y_pred.max())
plt.plot([low, high], [low, high], "r--")
plt.xlabel("Actual Price"); plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Airbnb Prices"); plt.tight_layout(); plt.show()

# 9. Residual plot
residuals = y_test - y_pred
plt.figure(figsize=(6, 5))
plt.scatter(y_pred, residuals, alpha=0.35)
plt.axhline(0, linestyle="--", color="red")
plt.xlabel("Predicted Price"); plt.ylabel("Residual")
plt.title("Residual Plot"); plt.tight_layout(); plt.show()

# 10. Compare error for lower- and higher-priced listings
results = pd.DataFrame({"actual": y_test, "predicted": y_pred})
results["absolute_error"] = (results.actual - results.predicted).abs()
median_price = results.actual.median()
results["price_group"] = np.where(results.actual <= median_price,
                                  "Lower-priced", "Higher-priced")
group_mae = results.groupby("price_group")["absolute_error"].mean()
print("\\nMAE by price group:\\n", group_mae)
group_mae.plot(kind="bar", title="Prediction Error by Price Group")
plt.xlabel("Price Group"); plt.ylabel("Mean Absolute Error")
plt.tight_layout(); plt.show()
```